Coflect HITL: Non-Blocking Human-in-the-Loop Training

Asynchronous Live Explainability for Fast Training Loops

Shivam Bharadwaj
Coflect

March 2026

Abstract

This report describes the technical design of **Coflect HITL**, a human-in-the-loop deep learning framework built around one central systems constraint: interactive explainability must not stall the training loop. Coflect isolates heavy explainability and intervention scheduling into asynchronous workers, while the trainer emits only compact scalar and metadata events. The system integrates live attribution overlays, deterministic text/ROI feedback, and top-k failure forecasting under explicit compute budgets. The implementation is Torch-first with an operational TensorFlow/Keras path and a thin backend adapter interface for future frameworks [4, 2, 3]. In a documented long-run benchmark on CPU synthetic workload, minimal Coflect instrumentation remains within the targeted overhead envelope (mean elapsed-time delta -0.37%, mean steps/sec delta $+0.27\%$), supporting the practical claim that live HITL controls can be integrated with near-zero training-throughput penalty when heavy work is decoupled.

1 Introduction

Human-in-the-loop training is often proposed as an answer to late-stage model debugging, but many implementations are either offline (post hoc only) or operationally expensive during training. Coflect HITL is designed as an engineering system for continuous intervention during optimization, not as a visualization afterthought.

The design target is explicit:

1. training throughput should not regress materially because a user opens the dashboard,
2. heavy explainability must never run on the trainer hot path,
3. trainer-worker communication should use small, stable JSON messages,
4. the UI should receive only scalars, compact metadata, and compressed attribution images.

This report documents the architecture and implementation choices that operationalize those targets.

2 Design Requirements and Non-Negotiable Constraints

Coflect HITL is built around four hard constraints:

1. **Do not slow training with visualization.**

2. **Do not block training on XAI.** Attribution generation must run in a separate process and can run on a separate device.
3. **Avoid trainer-side synchronization pressure.** Any expensive post-processing is shifted out of trainer compute steps.
4. **Restrict UI payloads.** UI payloads are limited to scalar metrics, small metadata, and compressed attribution images.

These constraints influence process boundaries, communication contracts, and the choice of deterministic feedback parsing instead of a generative in-loop interpreter.

3 System Architecture

3.1 Runtime Topology

Coflect HITL runs as a multi-process topology:

- **Backend (FastAPI):** event ingest, WebSocket broadcast, feedback store, backend-scoped XAI queue, backend-scoped forecast queue [1].
- **Trainer (Torch or TensorFlow):** performs optimization, emits compact metrics, enqueues XAI requests, emits compact forecast telemetry, polls feedback periodically.
- **XAI worker:** loads latest snapshots asynchronously, computes LiveCAM/Grad-CAM/SmoothGrad overlays, publishes compressed PNG payloads.
- **Forecast worker (CPU-only):** ranks likely future failures and controls intervention-window opening based on gating logic.
- **UI:** static dashboard served by backend at `/`, receiving live events over WebSocket and posting user feedback over REST.

3.2 Control Plane vs Data Plane

Coflect separates traffic classes:

- **Control-plane events** (small JSON): metrics, queue requests, feedback state transitions, forecast gates, worker status.
- **Visualization payloads** (compressed): attribution overlays as `png_b64` only, generated out-of-process.

No raw high-volume tensor streams are pushed directly from trainer to UI.

3.3 Message Contracts

Key event/request payloads are schema-like dataclasses (e.g., `XaiRequestPayload` with `step`, `sample_idx`, classes, risk metadata, backend tag). Wire helpers are intentionally fail-soft: network errors are logged at debug level and do not crash trainer execution.

4 Trainer-Side HITL Mechanism

4.1 Deterministic Text Instruction Parsing

Free-text instructions are parsed by a deterministic regex/rule parser (no in-loop LLM calls):

- absolute focus strength commands (e.g., `focus=0.3`),
- signed strength deltas (increase/decrease by percent or scalar),
- ROI commands via normalized coordinates (`x0,y0,x1,y1`), rectangle form (`x,y,w,h`), and region keywords (center, top-left, etc.).

This yields reproducible policy updates and removes nondeterministic control from the optimization loop.

4.2 Feedback Application and Pause/Resume

Trainer polls backend feedback at configurable cadence (`feedback_poll_every`). Feedback can:

- set `paused=true/false`,
- set/adjust `focus_lambda`,
- define ROI box or dense `roi_mask`,
- target sample-specific rules (bounded map, currently capped to 10 entries).

Applied state transitions are emitted as `trainer_feedback_applied` events for auditability.

4.3 Auxiliary ROI Objective (Cheap by Construction)

The auxiliary objective is intentionally sparse:

- computed every `aux_every` steps,
- evaluated on a small subset (`aux_subset`),
- only active when `focus_lambda>0` and an ROI policy exists.

For both Torch and TensorFlow trainers, masked inputs are generated from ROI policy and routed through an additional loss pass:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{primary}} + \lambda_{\text{focus}}\, \mathcal{L}_{\text{aux}}. \tag{1}$$

By making cadence and subset explicit, this channel remains intervention-capable but computationally bounded.

4.4 Asynchronous Snapshot Sync

Trainer snapshots are written by a background thread (queue size 1) using atomic temp-file replace:

- Torch: `model_step_{step}.pt`
- TensorFlow: `model_step_{step}.npz`

Queue size 1 intentionally drops stale snapshots, preserving newest-state availability while minimizing trainer-side I/O pressure.

5 Live Explainability Pipeline (LiveCAM)

5.1 XAI Worker Lifecycle

Each XAI worker loops over backend queue dequeue calls:

1. dequeue compact XAI request,
2. load latest snapshot if newer than currently loaded model step,
3. regenerate input sample by `sample_idx`,
4. compute attribution,
5. emit `xai_image` and `xai_timing` events.

This design fully decouples attribution from trainer step execution.

5.2 Attribution Methods

LiveCAM worker supports method selection:

- `gradcam/livecam`: class activation map [5],
- `smoothgrad`: averaged noisy input-gradient saliency [7],
- `consensus`: normalized blend of Grad-CAM and SmoothGrad.

Outputs include a heatmap overlay and derived metadata:

- `xai_agreement` between Grad-CAM and SmoothGrad,
- `focus_bbox` (normalized),
- top class probabilities,
- per-request XAI latency `xai_ms`.

5.3 Budget Guardrail

Workers enforce a rolling attribution budget using a 60-second sliding window over measured attribution times:

$$\sum_{(t_i,m_i)\in\mathcal{W}_{60s}} m_i \leq B_{\text{ms/min}}. \tag{2}$$

If budget is exhausted, request execution is skipped and `xai_budget_skip` is emitted. This bounds explainability compute without touching trainer loop timing.

6 Forecast Worker and HITL Window Scheduling

6.1 Why Forecast-Gated Intervention

Instead of showing arbitrary wrong predictions, Coflect ranks candidates likely to remain problematic over the next horizon and opens intervention windows only when training has matured sufficiently.

6.2 Telemetry and Per-Sample State

Trainer sends compact telemetry samples (e.g., p_{true}, margin, correctness, predicted class, sample index) at configurable cadence. Forecast worker maintains rolling per-sample histories and epoch-level accuracy counters.

6.3 Risk Function

For each sample with sufficient history, risk is computed as:

$$\begin{aligned}\text{risk} = 0.40\,\text{err_rate} + 0.25\,(1 - \overline{p}_{true}) + 0.15\,(1 - \widetilde{m}) \\ + 0.15\,\text{flip_rate} + 0.05\,\text{stagnation},\end{aligned} \tag{3}$$

where $\widetilde{m}$ is normalized margin and stagnation increases when confidence trend worsens. Scores are clamped to $[0, 1]$.

6.4 Intervention-Window Gates

A review window opens only when all gates pass:

- warmup passed,
- competence above chance + margin,
- accuracy plateau condition,
- residual error floor remains,
- top-k candidate stability overlap,
- cooldown after last intervention.

Window transitions emit explicit `hitl_window` events. When open, forecast candidates are proactively enqueued as `request_kind=forecast` XAI requests.

7 Backend Abstraction and Extensibility

Coflect defines a minimal backend adapter contract with four methods:

`forward`, `loss`, `step`, `attribution`.

Current state:

- **TorchAdapter**: production path.
- **TensorFlowAdapter**: MVP runtime path integrated with trainer and XAI worker.
- **JaxAdapter**: scaffold interface pending integration tests.

This keeps module logic backend-agnostic while allowing backend-specific performance tuning under a stable contract.

8 Performance Characteristics

8.1 Benchmark Setup

A documented long-run benchmark artifact is included in the repository benchmark folder (timestamped 2026-02-28 UTC), using CPU synthetic workload with 200-step runs and repeated measurements [6].

8.2 Observed Overhead

Configuration	Mean Elapsed (s)	Mean Steps/s
Baseline trainer	105.8076	1.8934
Coflect minimal instrumentation	105.4117	1.8985

Table 1: Reported benchmark summary from bundled artifact.

Derived comparison in artifact:

- elapsed-time delta: -0.374%,
- steps/sec delta: $+0.268\%$.

Interpretation: for the measured setup, minimal HITL instrumentation is effectively overhead-neutral. This aligns with the design objective of preserving training throughput when heavy paths are disabled or offloaded.

8.3 Why Throughput Impact Stays Low

Three implementation choices dominate:

1. asynchronous snapshot writer with stale-drop queue,
2. small JSON-only trainer emissions,
3. separate-process explainability and CPU-only forecasting.

9 Operational Interface

9.1 CLI and Notebook Modes

Coflect supports two practical workflows:

- one-command launcher (backend + trainer + workers),
- notebook-native bridge that can auto-start sidecar services and preserve research workflow structure.

9.2 Event Observability

Event types include metrics, queueing events, snapshot queue notifications, feedback application records, window transitions, and XAI timing. This creates an operational trace useful for post-mortem and reproducibility.

10 Limitations and Current Boundaries

- TensorFlow/Keras path is marked experimental in the support policy.
- JAX backend is currently scaffolded but not yet integrated end-to-end.
- Current benchmark evidence is synthetic CPU; broader real-dataset GPU benchmarks are still required.
- LiveCAM quality remains attribution-method dependent; agreement metrics are provided, but interpretation still requires human judgment.

11 Roadmap

Near-term engineering priorities:

1. extend backend parity (Torch and TensorFlow feature convergence, then JAX integration),
2. strengthen dataset-aware notebook-to-worker coupling in packaged deployments,
3. expand benchmark matrix across hardware classes and real datasets,
4. introduce more modality-aware attribution decomposition for multimodal models.

12 Conclusion

Coflect HITL demonstrates a systems-first approach to interactive training: maintain training-loop simplicity, offload heavy explanation and forecasting, and preserve deterministic operator control. The architecture shows that live explainability, forecasted intervention windows, and text/ROI feedback can coexist with near-zero throughput regression when process boundaries and message contracts are designed explicitly for performance.

References

[1] Fastapi documentation. https://fastapi.tiangolo.com/, 2026. Accessed March 2026.

[2] Martín Abadi, Ashish Agarwal, Paul Barham, et al. Tensorflow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation*, 2016.

[3] James Bradbury, Roy Frostig, Peter Hawkins, et al. Jax: composable transformations of python+numpy programs. https://github.com/google/jax, 2018.

[4] Adam Paszke, Sam Gross, Francisco Massa, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, 2019.

[5] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, et al. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *IEEE International Conference on Computer Vision*, 2017.

[6] Shivam Bharadwaj. Coflect hitl overhead benchmark artifact. Repository artifact: docs/benchmarks/hitl_overhead_longrun_2026-03-01.json, 2026.

[7] Daniel Smilkov, Nikhil Thorat, Been Kim, Fernanda Viégas, and Martin Wattenberg. Smoothgrad: removing noise by adding noise. *arXiv preprint arXiv:1706.03825*, 2017.